

Chapter 9

A Multi-thread GRASP_xELS for the Heterogeneous Capacitated Vehicle Routing Problem

Christophe Duhamel, Christophe Gouinaud,
Philippe Lacomme, and Caroline Prodhon

Abstract. This chapter focuses on the definition of an efficient parallel metaheuristic which takes advantage of the multi-core design of recent processors. The approach is designed as a Greedy Randomized Adaptive Search Procedure (GRASP) hybridized with a multi-threaded version of an Evolutionary Local Search (ELS) metaheuristic scheme. Our approach is evaluated on an extension of the Vehicle Routing Problem where a heterogeneous fleet of vehicles is available to service a set of customers. The objective consists in designing a set of trips for a limited heterogeneous fleet of vehicles located at a depot node which minimizes the total transportation cost. Each type of vehicles is defined by a capacity and by the number of available vehicles. The efficiency of the parallel approach is evaluated on a new set of real-life instances built out of data from the French districts. A fair comparative study, using a same implementation, is done to evaluate the impact of the number of threads on the convergence rate. Thus, a better trade-off between solution quality and computational time can be reached. The numerical experiments show that the hybrid GRASP_xparallel ELS outperforms the classical iterative version and provides numerous new best solutions.

9.1 Introduction

The design of parallel implementations of metaheuristics has received a considerable amount of attention in the last two decades with the development of new hardware

Christophe Gouinaud · Philippe Lacomme
Laboratoire d'Informatique (LIMOS, UMR CNRS 6158), Campus des Cézeaux,
63177 Aubière Cedex, France
e-mail: {christophe.gouinaud,placomme}@isima.fr

Christophe Duhamel · Caroline Prodhon
Institut Charles Delaunay (LOSI) and STMR (UMR CNRS 6279),
Université de Technologie de Troyes, BP 2060, 10010 Troyes Cedex, France
e-mail: christophe.duhamel@isima.fr, caroline.prodhon@utt.fr

technologies which provide several efficient and cheap opportunities. This chapter focuses on the definition of a Greedy Randomized Adaptive Search Procedure (GRASP) hybridized with a multi-threaded version of an Evolutionary Local Search (ELS) metaheuristic scheme which takes advantage of the multi-core design of the recent processors. Our approach is tested on the Heterogeneous Vehicle Routing Problem (HVRP), an extension of the classical Vehicle Routing Problem (VRP) where a heterogeneous fleet of vehicles is available to service a set of customers.

A brief reminder of the recent technological advances in parallel implementations of algorithms is presented in the next Section, with a state of the art on parallel metaheuristics. Then, Section 9.3 introduces the problem under consideration in this chapter, the HVRP. The parallel hybrid framework is stated in Section 9.4. Its implementation for the HVRP is explained in Section 9.5. Then, the details of the components of the method are exposed in Section 9.6. Finally, a computational evaluation of the method is proposed through Section 9.7 before concluding remarks.

9.2 Parallel Metaheuristics

Before presenting our hybrid metaheuristic based on a GRASP and a parallel implementation of an ELS scheme, this first section gives an overview of the available technologies and of the state of the art to justify our choice of a multi-thread implementation to solve the HVRP.

9.2.1 *Technologies*

Parallel computing environment offers various ways to implement parallelization. More specifically, three main trends can be identified.

Historically, the first trend is based on computer cluster architectures and software environment. It relies on communication frameworks as MPI (Message Passing Interface) and it usually requires a large communication bandwidth in order to stay efficient. Such an approach provides a master-slave message passing paradigm but it also requires a huge financial investment in order to build the cluster and the high speed communication network.

The second trend is more recent. It consists in taking advantage of the capabilities of the graphics processors included in modern computers (Graphics Processing Unit - GPU). Even if they were first developed to afford the computing requirements of the visual effects in modern 3D games, it has been lately evidenced they could be used for other purposes. For instance, NVIDIA provides a CUDA (Compute Unified Device Architecture) library with a user-friendly C++ interface in order to simplify the development of parallel algorithms. Their GPU can handle a huge number of threads to be used but they are unfortunately gathered in blocks with a slow shared memory. Depending on the GPU capability, the dedicated memory is

limited to several kilobytes. This is a strong limitation, especially when one needs to implement Operations Research algorithms in which the amount of required data is large. Note also that GPU frequency is quite low when comparing to modern CPU (Central Processing Unit). In the same trend, a new generation of parallel computing architectures (as in Tesla workstations), designed by NVIDIA as well, delivers fast data-parallel processing. It is however limited to dedicated and professional complex supercomputing problems due to the price of such workstations.

The third trend consists in using the multi-core technologies available on every recent processor to address the parallelization challenge. Those CPU typically offer less parallel threads (from 2-cores for basic processors to 8-cores for high-end processors, and even more on specific machines) than NVIDIA GPU. However, it allows a high speed memory and higher frequencies. In addition, the large data accesses are convenient for Operations Research algorithms. These reasons prompt us to develop a framework based on this trend even if the parallelization is limited compared with the number of parallel threads offered in NVIDIA technology.

9.2.2 State of the Art for Operations Research Problems

Various optimization problems have been addressed for years using parallel metaheuristics. Literature is scarce and also confusing as papers address a wide range of problems, propose different approaches and different criteria are used to analyze the parallelization efficiency.

Experiments in parallelizing metaheuristic for Operations Research problems encompass both population-based metaheuristics, such as Genetic Algorithms, and single solution-based metaheuristics including Tabu Search and GRASP (Greedy Randomized Adaptive Search Procedure) for instance. The interest mainly comes from many real-life applications, from scheduling to vehicle routing, when size is too large to get high quality solutions in a reasonable time for sequential approaches. Fiechter in 1994 [15] developed an efficient parallel Tabu Search algorithm for the Traveling Salesman Problem (TSP), implemented on a network of transputers in OCCAM language (concurrent programming language working on Communicating Sequential Processes - CSP). The author provided a thorough analysis of the speed-up, *i.e.* the ratio of the average CPU time using one transputer and p transputers. He also introduced a careful description of the algorithmic key points which includes, among others, a diversification and an intensification strategy. Ten years later, Berger and Barkaoui [24] introduced a parallel Hybrid Genetic Algorithm (HGA) for the Vehicle Routing Problem with Time Windows (VRPTW) based on the simultaneous evolution of two populations of solutions focusing on separate objectives and subject to time windows constraints relaxation. The implementation uses the PVM (Parallel Virtual Machine) library and the parallel procedure has been implemented in C++ on a cluster of 19 computers. The master process manages the execution of the algorithm at a high level. It synchronizes atomic genetic operations and handles the parent selection process. The slave processes concurrently

execute the reproduction and the mutation operators. Numerical experiments are based on the medium-scale instances of Solomon. They include 56 instances with up to 100 nodes. The authors provide a performance analysis of the parallel HGA and a comparison with previous published methods. Their approach is proven to be time efficient, cost effective and thus it is competitive as it matches the performance of best-known heuristic routing procedures. Moreover RHGA provides six new best-known solutions.

The Vehicle Routing Problem (VRP) is also addressed by Le Bouthillier and Crainic [2] using a solution warehouse strategy, in which several search threads cooperate by asynchronously exchanging information on the best solutions found so far. The exchanges are performed through a mechanism, denoted solution warehouse, which holds and manages a pool of solutions. The asynchronous communication ensures independence of the individual search processes. Each search process implements a different metaheuristic including an Evolutionary Algorithm and a Tabu Search procedure. The results obtained on an extended set of test problems show that the parallel procedure achieves linear accelerations. This approach identifies solutions of comparable quality to those obtained by the best sequential methods in the literature. Caricato et al. [36] propose a parallel Tabu Search for the pickup and delivery problem, *i.e.* a special VRP where each request (product) has to be transported from an origin node to a destination node by means of vehicles. The authors introduce two sequential heuristics and a parallel Tabu Search. The method is run on a cluster of 4 PC, each one including two Pentium III processors. The code is implemented in C using the MPI library. The Periodic VRP (PVRP) is addressed by Drummond et al. [32] with a parallel Genetic Algorithm (PGA) and local search heuristics. The PGA relies on the Island model and includes periodic migration between islands (local pools) with low frequencies. Experiments are done on a workstation with 4 RISC System/6000 processors and the methods are implemented in C using MPI for parallelism. Numerical experiments let authors conclude that the parallel implementation provides significant advantages not only in terms of running time but also in terms of search quality. The Heterogeneous VRP (HVRP) has been addressed in 1998 by Ochi et al. [33] using a parallel Evolutionary Algorithm. The authors combine a parallel Genetic Algorithm with a Scatter Search and they apply decomposition procedures (petals decomposition).

Bortfeld et al. [1] introduce a parallel Tabu Search Algorithm for the container loading problem. They propose a distributed parallel approach based on the concept of multi-search threads developed by Toulouse et al. [35]. Several search paths are explored in a concurrent way: cooperation is achieved by the exchange of solutions at the end of the search phases. The parallel search processes are run on LAN (Local Area Network) workstations. The efficiency of the parallel Tabu Search Algorithm (TSA) is evidenced by an extensive comparative test including classical instances. According to their test, the container volumes are already loaded at a high rate in the solutions provided by the sequential TSA. Only slight improvements are achieved by the parallelization. The authors also note that the communication between the TSA processes weakly contribute to this effect, which corroborates previous similar results. For instance, Crainic et al. [47] proposed the parallelization of a TSA for

solving a warehouse location problem. They show that the best results were obtained without communication between the concurrent processes. Previous experiences in the field make them consider the parallelization of a method as a relevant methodical extension, especially if other concepts for the improvement of a sequential method are already exhausted. They conclude that such an approach provides only limited enhancements in terms of solution quality.

Parallel hybrid metaheuristics have been proposed for the flexible job shop problem by [26] including two major modules: the machine selection module is sequential while the operation scheduling module is run in parallel. Numerical experiments are provided using a NVIDIA GPU Tesla C870 GPU (512 GFLOPS) with 128 streaming processors cores. They use benchmark problems from literature. The Hybrid Flow-Shop problem with multiprocessors tasks scheduling has recently been addressed by [26] using a parallel greedy algorithm approach. The algorithm consists on two phases: a destruction phase and a construction one. A similar problem has been recently addressed by Bozejko [51].

The sequential ordering problem is addressed in a parallel environment by Guerriero and Mancini in 2003 [20]. They use a parallel version of the rollout algorithm first proposed by [17]. It is based on a multi-thread parallelization strategy where different regions of the solution space are explored by different threads. Information exchange is done periodically. The implementation is done on a cluster of PC and the authors conclude that parallelization both speeds up the convergence and the solution quality. Lately, in 2009, the quadratic assignment problem has been addressed by James et al. [45] using a Cooperative Parallel Tabu Search (CPTS). The numerical experiments demonstrate the benefits that can be obtained with parallel computing in terms of convergence rate and of solution quality. CPTS provides several new best solutions. A parallel Scatter Search metaheuristic for solving the feature subset selection problem in classification is introduced by [22]. Two methods are proposed to combine solutions in the Scatter Search (SS). The parallelization is done by simultaneously running the two combination methods. The parallel Scatter Search achieves values similar to both sequential SS algorithms, but uses a smaller subset of features. Moreover, the parallel algorithm is more accurate than sequential algorithms.

Speedups can strongly vary since they depend on the problem and on the metaheuristic used for parallelization. Sometimes a slight improvement in terms of solution quality can be achieved while the computational time remains stable. However there are many ways to perform a relevant parallelization. For instance, Ribeiro and Rosseti [13] investigate parallel strategies for GRASP metaheuristics. The experiments they did for the 2-path Network Design Problem show that a speedup about 17 is obtained with the 32-processor cluster. Our aim is to provide a parallel method that can be run on computers earned in any compagny, *i.e.* without investing in expensive machines. Thus our choice goes to a multi-threaded strategy that takes advantage of the multi-core processors available on any recent computer.

Independently to the parallel technology used and whatever the problem and the metaheuristic used, the state of the art enable us to name key points for efficient parallelization. They include the way to perform diversification and intensification,

and the cooperation mode between threads. These three points are considered in our approach as summarized in Table 1 from Section 9.4.3. Before detailing them, next Section exposes the problem used to attest the performance of our GRASP hybridized with a multi-threaded ELS.

9.3 Heterogeneous Capacitated Vehicle Routing Problem

The Capacitated Vehicle Routing Problem (CVRP) is a standard NP-hard node routing problem which has received considerable attention in the last decades, see [40, 39, 12] for instance. It consists in optimizing the delivery of goods required by a set of customers. It can be fully defined by considering a depot and a set of n customers which correspond to the nodes of a complete graph $G = (V; E)$. V is the set of $n + 1$ nodes, 0 being the depot and $1...n$ being the customers. Each edge $e \in E$ has a finite nonnegative routing cost $c_e \geq 0$ and each node $v \in V \setminus \{0\}$ has a demand $d_v \geq 0$. A fleet of homogenous vehicles of limited capacity Q is based at the depot. The objective is to design a set of trips of minimal total cost to service all customers. A trip is a circuit performed by one vehicle, starting at the depot, sequentially visiting a subset of nodes and returning to the depot. The total trip load must not exceed the vehicle capacity Q . Since split deliveries are not allowed, each customer is serviced by exactly one vehicle. As stressed in [26], efficient resolution of medium to large CVRP instances is currently limited to metaheuristics.

In its basic formulation, the CVRP is not realistic enough to accomodate real features coming from companies. In this context, many extensions have been considered during the last decades. Thus, keeping track of CVRP development is really difficult as node routing problems transcends several academic disciplines. In 2008, Eksioglu et al. [5] have provided a methodology to classify the literature of the VRP, *i.e.* a taxonomic framework. Their proposal extends the previous proposal of Current and Marsh in 1993 [29]. Additional VRP constraints can be classified into three sets: scenario characteristic, physical characteristics and information characteristics. Scenario characteristics include, for instance, customer service demand quantity (deterministic, stochastic), load splitting constraints (splitting allowed or not), time windows (soft time windows, strict time windows), time horizon (single period, multi period) or customer types (linehaul, backhaul, transfer). Physical characteristics include the number of origin points (single or multiple origins), the time window types (restrictions on customers, on roads), the number of vehicles (exact number of vehicles, limited number of vehicles, and unlimited number of vehicles). Information characteristics include the evolution of information (static or partially dynamic) or the quality of information (stochastic, deterministic).

In this chapter, we consider the deterministic case in which a company owns a limited heterogeneous fleet of vehicles. This is an extension of the Vehicle Fleet Mix Problem (VFMP) for which the fleet is composed of an unlimited number of vehicles of several types. In VFMP, each type k is defined by a vehicle capacity Q_k , a fixed cost f_k , a variable cost v_k and a travel cost between pairs of nodes that can

be defined by a distance matrix (VFMP-F), by a variable (VFMP-V) or by a mix of both (VFMP-FV). A trip of length L performed by a vehicle of type k has a cost $f_k + L.v_k$. The goal is to compute a set of trips and to assign vehicles to trips to minimize the total cost. The VFMP typically appears in situations where the fleet is not yet purchased: it combines tactical decisions (selecting the number of vehicles to be acquired) and operational decisions (computing the trips and the vehicles assigned to them). The Heterogeneous Fleet VRP (HFVRP or HVRP) shares the same structure as the VFMP with a limited availability a_k for vehicles of type k . Both the VFMP and the HVRP are NP-hard, since they generalize the VRP.

The fleet composition induces a trip assignment to vehicle type and a limitation on resources for the HVRP case. Thus, the latter is more restrictive. Published methods are, in most of the cases, dedicated to only one of the three VFMP versions or to the HVRP. For instance, two articles have recently considered the HVRP only: Li et al. [21] introduce a Memory Programming metaheuristic and Brandão [4] proposes a Tabu Search algorithm. Prins [12] is the first to propose a single approach to handle all the non-homogeneous fleet VRP versions. It is based on a Memetic Algorithm specially tuned for each version of either the VFMP or the HVRP. Besides, he presents a thorough review of the literature on the history of VFMP and HVRP. Duhamel et al. [9] also handle both the VFMP and the HVRP. Besides, they propose 96 new realistic instances based on data from the real French counties.

Results provided by state-of-the-art methods on “classical” HVRP instances (Taillard’s instances) are usually very close to the best-known solutions, and they are obtained within a reasonable amount of time. Thus, these instances cannot be used to compare the methods anymore. The situation is different on the recent large instances proposed by Duhamel et al. [9]¹. To the best of our knowledge, they are the only recent large instances available. The aim of this chapter is to emphasize the interest of a new parallel implementation of ELS to achieve good solutions and to evaluate the performance on such large instances.

9.4 Hybrid GRASP x Parallel ELS

In this chapter, we propose a hybrid metaheuristic that focuses on the third parallelization trend, *i.e.* taking advantage of the multi-thread capabilities of the modern multi-core processors. This choice is motivated by two characteristics:

- first, algorithms dedicated to Operations Research problems typically require large amount of data and can benefit from shared-memory machines;
- second, our goal is to propose a threaded cooperative search strategy to better explore the solution space through a functional parallelism. In this context, fine-grained application is not much useful and parallel programming languages are

¹ <http://www.isima.fr/~duhamel>, <http://www.isima.fr/~lacomme/hvrp/hvrp.html> or <http://prodhonc.free.fr/>

not well-suited. An information synchronization done at a low or medium frequency (medium- or coarse-grained application) is sufficient.

To achieve an efficient cooperative search, we propose a clever parallelization of ELS embedded within a GRASP.

9.4.1 *GRASP×ELS Principle*

ELS (Evolutionary Local Search) is an extension of ILS (Iterated Local Search). ILS was introduced by Lourenço et al. [16]. It consists in applying a perturbation to the current solution S before improving the resulting solution S' through a local search. The new solution becomes the new current solution. Note that the perturbation is similar to the mutation operator found in Genetic Algorithms.

ELS was enhanced by Prins in 2009 [11] for routing problems. It is similar to ILS except that *nd* “children” are generated from S at each iteration. Thus, each child is a copy of S on which a mutation, then a local search, are applied. The best child is selected to become the new ongoing solution S . The purpose of ELS is to better explore the current local optimum neighborhood before leaving it.

In addition, to ensure some diversity along the search space exploration, a multi-start scheme can be implemented. Using any randomized process ensuring diversity, a set of initial solutions is built. ELS is applied to each initial solution generated in this set. In this chapter, such a scheme is applied with some diversification brought by a metaheuristic called GRASP (Greedy Randomized Adaptive Search Procedure) developed by Feo and Resende [46].

A classical GRASP generates a solution thanks to a greedy randomized heuristic. This solution is improved by a local search and the process is re-iterated until a stopping criteria is reached. The best solution encountered during the iterations is saved and returned at the end. In the hybrid version, the local search is replaced by ELS, leading to the hybrid GRASP×ELS [11]. Both GRASP×ELS and multi-start ELS have proven to be highly efficient on some optimization problems, including but not limited to, VRP [11] and LRP [8].

9.4.2 *Parallelization*

The proposed hybrid metaheuristic is based on the GRASP×ELS scheme. It is parallelized on the ELS part to bring several concurrent searches in the solution space attempting to explore even wider regions of the search space. Thus, it belongs to type 3 according to Guerriero’s classification [20]: several search threads that simultaneously perform an ELS with a given degree of synchronization and cooperation. The whole framework we propose is illustrated in Figure 9.1 and the details of the components are exposed in Section 9.6.

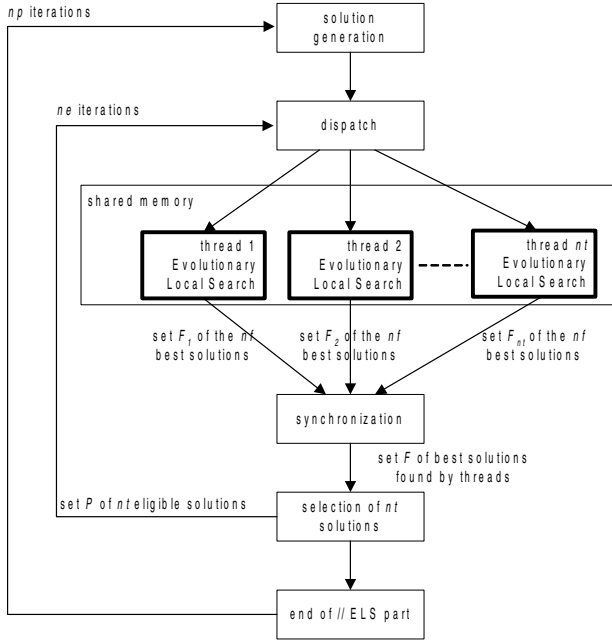


Fig. 9.1 Multi-start parallel ELS principle

9.4.3 Key-Features of the GRASP *x* Parallel ELS

Increasing the number of threads allows more evaluations to be performed. However, this may not help improving the performances if no further bright strategy is implemented. Our goal is first directed towards achieving a good speed-up while keeping a natural and reasonable parallel implementation effort. Thus, the proposed parallelization relies on an asynchronous cooperative search thread strategy (as seen in Figure 9.1). On the other hand, a wrong equilibrium between diversification and intensification may also lead to a premature convergence and trap the method on a local optimum.

An equilibrium between diversification, intensification and cooperation is important and it is ensured by all the key-features we promote within our GRASP *x* parallel ELS, which are:

- the greedy randomized heuristic through the GRASP scheme;
- the assignment of an initial solution to each thread through a *Dispatch* procedure;
- the parallel ELS scheme;
- the synchronization of the threads;
- the selection of the solutions to be dispatched;
- the shared memory management.

Table 9.1 summarizes the impact of those features on the search process. The next Section presents an effective exploration strategy, adapted to the HVRP. Then Section 9.6 details the key features

Table 9.1 Parameter impact in diversification, cooperation and intensification

Key features	Diversification	Cooperation	Intensification
GRASP	X		
Dispatch	X		X
Parallel ELS	X		X
Synchronization		X	
Selection	X		
Shared Memory Management	X	X	X

9.5 GRASP x Parallel ELS for the HVRP

The solution space exploration should be proficient for the problem under consideration to ensure the best possible efficiency. We consider here vehicle routing problems, and more specifically the heterogeneous VRP. An exploration that has been proven effective for routing problems is based on the alternation between two solution representations: solutions encoded as giant tours (TSP tours on the n customers) and solutions encoded as sets of trips (see Figure 9.2).

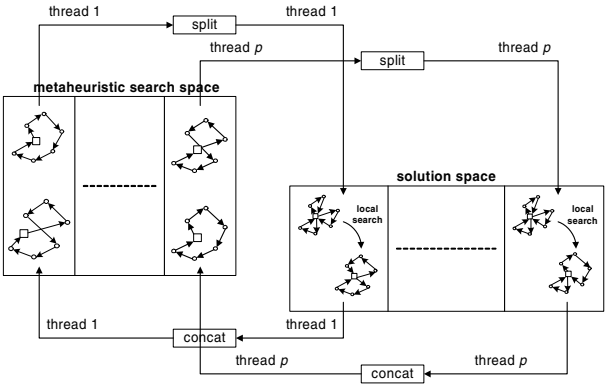


Fig. 9.2 Combination of the two search spaces

The alternation between solution representations is based on the following principle:

- **Split:** a giant tour T , made of the n customers to visit, is transformed by a splitting procedure into a HVRP solution S with respect to the given sequence and

the vehicle fleet composition. This solution S can then be improved by a local search. It can also be perturbed or left unchanged.

- **Concat:** a procedure transforms S into a giant tour T' by concatenating its trips.

The giant tour T' can be split again in order to get a new HVRP solution. This process allows alternation between the giant tours space and the HVRP space.

Splitting was originally proposed by Beasley in 1983 [27] as the second phase of a route-first cluster-second heuristic to solve the VRP. It has been successfully applied to many routing problems within general optimization frameworks. A state of the art [9] about Split approaches in routing problems surveys more than 40 recent publications on the topic. It shows this strategy has proved its efficiency and received a considerable amount of attention especially since 2001. It covers a large area including node routing problems, arc routing problems and more complex routing problems with additional constraints as the Capacitated Arc Routing Problem (CARP [38]), the Vehicle Routing Problem (VRP [10]) and the Location Routing Problem (LRP [7, 8]). Moreover, the high quality solutions obtained by Prins [10] are a suitable indication of the interest of such an approach. Our GRASP x parallel ELS belongs to this line of research.

The transformation from T to S through the Split procedure proposed by Prins [10] consists first in building an auxiliary acyclic graph H based on the sequence of customers given by T . The auxiliary graph is composed of $n + 1$ nodes numbered from 0 to n , 0 being the depot and $T(i)$ being the customer at position i in the sequence. An arc from node i to j corresponds to a subsequence $u_{ij} = (T(i + 1), \dots, T(j))$ of T . The quantity to collect along u_{ij} is denoted as $Q(u_{ij})$. Thus, the trip u_{ij} consists in a route starting from a depot and going to node $T(i + 1)$, from $T(i + 1)$ to $T(i + 2)$ and so on until $T(j)$ before coming back to the departure depot node. Moreover, all the additional problem constraints have to be satisfied by u_{ij} for the arc (i, j) to belong to the graph. Then, optimally splitting T into routes corresponds to computing the min-cost path from node 0 to node n in H . Since the HVRP includes some resource constraints as the vehicle types (to each type is given a limited number of available vehicles), several labels per node have to be handled to compute the shortest path in H [34]. Thus, several strategies to perform the label propagation can be used. Since the graph is acyclic, the original strategy consists in iteratively considering the nodes from 0 to $n - 1$ to generate the labels in a greedy way. A recent variation [9] uses a depth first search exploration of the graph to reach the final node as fast as possible in order to get sharp bounds. The second strategy is used in this chapter.

When dealing with resource constraints, the number of labels per node generated by Split in H can be large [9]. A solution consists in limiting both the maximal number of labels per node ($mlpn$) and the total number of labels generated during the whole split procedure (mtl). The Split procedure becomes suboptimal since it may discard the optimal label, but its computational time is greatly reduced. Thus, it becomes compatible with an iterative metaheuristic scheme. The Split procedure depends on these two parameters.

The alternation between solution representations as giant tours and HVRP solutions is applied in the core of the ELS we propose. At each iteration of the ELS, a *mutation* operator is applied. It aims at modifying the solution to explore its neighborhood. The obtained children are then improved by a *local search*.

For the HVRP, the *mutation* is defined on giant tour to generate new customer sequences. A child is obtained by swapping two randomly selected nodes in the giant tour. The resulting sequence is evaluated by the Split procedure and the local search is called. Such a customer exchange would mostly be ineffective on HLRP solutions, since the local search can cancel it in most cases. It is far more effective on giant tours, since a basic customer exchange may force Split to radically modify several trips.

The *local search* we implemented works on the HVRP solutions. It uses a first improvement strategy combining several classical VRP neighborhoods: 2-Opt within a trip, 2-Opt between two trips, Swap within a trip and Swap between two trips. This kind of approach is commonly used in routing problems including the CARP [37], the VRP [10], the HVRP [12] and the LRP [8].

9.6 Detailed Components of the GRASP x Parallel ELS

A detailed description of the key-features of the proposed method to solve the HVRP is given in this Section. They include:

- the greedy randomized heuristic through the GRASP scheme;
- the assignment of an initial solution to each thread through a *Dispatch* procedure;
- the parallel ELS scheme;
- the synchronization of the threads;
- the selection of the solutions to be dispatched;
- the management of the shared memory.

9.6.1 Initial Solutions of the GRASP

The *greedy randomized heuristic* is the first step of the method. It is called at the beginning of each iteration of the GRASP. It creates an initial solution S by generating a giant tour through a greedy randomized heuristic and splitting it into HVRP trips. Then, the resulting solution is improved by local search. Such generation was promoted by Prins in its genetic algorithm for the HVRP [12]: the initial population is made of random trips and of a limited number of high-quality solutions computed by randomly generating a giant tour and performing a local search on it.

Our greedy randomized heuristic is based on the nearest neighbor algorithm. From the current node i (initially the depot), the nearest customer j is identified, with c_{ij} the distance between them. Then, the randomized version looks for the set of nodes k such that $c_{ik} \leq frac * c_{ij}$, $frac$ being a given percentage. The procedure

chooses at random a customer s from this set and adds it to the giant tour. s becomes the new current node and the algorithm goes on until it remains no customer.

Once the giant tour is completed, Split is called to evaluate the corresponding HVRP solution S . However, due to resource constraints on the vehicle fleet, the procedure might fail to compute a feasible solution. In this case, a new call to the greedy randomized heuristic is called to generate a new giant tour. If after *maxTry* attempts no feasible solution is built, another procedure is called to build a giant tour T at random until the splitting of T provides a suitable solution S . Such a strategy seems to be a suitable trade-off since generalization of well-know heuristics (including Clarke and Wright [23] heuristic and Golden et al. [6] heuristic) is not easy due to the limited heterogeneous fleet of vehicles.

The solution S is then improved by the local search detailed in Section 9.5 and saved *nt* times in a pool P .

9.6.2 Dispatching: Threads Management

The *Dispatch()* procedure assigns an initial solution to each thread. These initial solutions are selected from the pool P of eligible solutions, according to a dispatching strategy R . Then, each ELS runs asynchronously on the threads.

It is possible to consider several dispatching strategies in order to promote diversification or, at the opposite, intensification, *i.e.* to focus on specific areas of the solution space. For instance, one can consider the following strategies:

- *BF* (Best First) consists in assigning the best solution in P to every thread: this dispatching strategy favours intensification;
- *DF* (Diversity First) consists in assigning one different solution to each processor: this dispatching strategy favours diversification.
- *CBF* (Couple Best First) consists in assigning the best solution to threads 1 and 2, the second best solution to threads 3 and 4 etc. Thus, this is a trade-of between *BF* and *DF*.

9.6.3 Parallel ELS Scheme

The algorithm below describes the *Parallel_ELS()* procedure in which each of the *nt* thread performs simultaneously one ELS.

Each thread starts an ELS with its assigned solution S . At the first iteration of the parallel part ($j = 1$) P contains only one solution. Thus, *Dispatch()* assigns the same initial solution to each thread. Then, *ns* ELS iterations, or *nr* unproductive ELS iterations, *i.e.* iterations without improving the incumbent solution, are performed (*while* loop lines 2-29).

The children generation (mutations and local search) is done lines 5-19. It produces *nd* children and updates the incumbent pair (S, T) accordingly. From T , the

Algorithm 10. Start_ELS_Thread

global parameters :
 ns : maximum number of iterations per ELS

 nd : number of children (mutations)

 nf : number of solutions to save

 nr : maximum number of iterations without improvement per ELS

 nc_max : maximal number of unproductive iterations during mutation

input parameters :
 T : initial solution

 i : thread identification

output parameters:
 F_i : vector storing the nb best solutions of the ELS in this thread i
 $r := 0, nc := 0, S := \text{call Split}(T)$
while ($u < ns$) **and** ($r < nr$) **do**
 $u := u + 1; f'' := \infty; j := 1$

// mutation loop

while ($j < nd$) **and** ($nc < nc_max$) **do**
 $T' := \text{call Mutation}(T)$
 $S' := \text{call Split}(T')$
if S' **is a solution** **then**

// HVRP feasible solution

 $S' := \text{call Local_Search}(S')$
if (S' **is not a clone**) **then**
 $T' := \text{call Concat}(S')$
if ($f(S') < f''$) **then**
 $f'' := f(S'); T'' := T'; S'' := S'$
 $j := j + 1$
else
 $nc := nc + 1$
else
 $nc := nc + 1$

// check the improvment

if $f'' \geq f(S)$ **then**
 $r := r + 1$ // update the number iterations without improvement

else
 $\quad$ Save T' into F_i //only keep the best nf solutions

// if a new best solution

if $f'' < f^*$ **then**
 $S^* := S''$ // update S^*
 $f^* := f''$
 $T := T''$ // the best ELS solution becomes the new initial solution

mutation produces a new giant tour T' . If splitting T' leads to a feasible solution S' , a local search is applied on it line 10. Then, an efficient clone detection (detailed in Section 9.6.5) is included to avoid thread search to be done on the same solution space area. Thus, if the S' is not a clone (line 11), it is considered for the next ELS iteration. Lines 13-15 are classical ELS steps: the best solution is saved into S'' . Note the value nc is increased if S' is unfeasible or if it is a clone. The value r is

increased line 22 each time an ELS iteration does not improve the incumbent solution. Otherwise S is considered to enter F_i , the ordered set keeping the nf best solutions found during the ELS. Lines 26-28 consist in updating S^* whenever required. Line 29 assigns T'' to T , providing the new starting giant tour required at line 2.

9.6.4 Synchronization and Selection Procedure

Parallel_ELS() simultaneously runs on the nt threads. However, each ELS can stop asynchronously. A **synchronization** is necessary before beginning a next parallel iteration. At this point, the *Selection()* procedure recover information from the threads by retrieving the set F_i of best solutions saved during each *Parallel_ELS()* and selecting a new pool P of nt solutions by considering both diversification and quality of solutions.

Diversity is important to avoid premature convergence as it favours exploration of new search space regions. Managing diversity requires the definition of a distance between solutions in an approach similar to what was defined in MAPM (Memetic Algorithm with Population Management) approach [31]. The distance depends on the representation of a solution. For binary problems, the Hamming distance can be used and several distances have been proposed for permutation problems [42, 43, 50]. The Wagner and Fischer's distance (the edit distance) is commonly used in many combinatorial problems [41, 30]. This way, a solution is stated as eligible if and only if it is far enough to the best found solution, or to a set of high quality solutions previously visited.

One can also consider that solutions of identical cost are quite similar in the sense they are at the same distance from the optimal solution, if one considers a cost-based distance. Thus, the cost-base distance has been considered sufficient to check if a solution must be kept into the set F of nf best solutions.

9.6.5 Shared Memory Management

The shared memory management is the last key feature of the framework. In fact, an efficient search strongly depends on the coordination between threads. The *Dispatch()* procedure does most of it, but one need another feature: the **clone detection** to avoid as much as possible costly and unprofitable solution evaluation. A clone can be defined as a solution that has already been visited.

Clones weaken the effectiveness of the parallel search and an option is to forbid clones to be considered during the search. Exact clone detection might be time consuming and even a basic comparison between two giant tours runs in $O(n)$. However, an efficient management be done by using hashing techniques [48]. Indeed, by associating each solution to a key which depends on the solution cost and on the number of trips ($key(S) = (S.Cost \times S.nbtrip) \bmod K$, with K a large integer), an

approximate and fast system can be implemented. Each key is stored in an array *Clone* such that *Clone*[*x*] gives the number of visited solutions whose key is *x*. Thus the clone detection can be done in $O(1)$. Using this array, one can control how many time the parallel ELS search are allowed to consider a previously visited solution.

However, this is not sufficient as several threads may consider the same solution. Thus, another policy has to be defined, leading to two ways to perform the detection:

- only consider solutions investigated by the thread;
- consider the whole set of solutions over the threads.

The latter policy increases the amount of communication between threads and favours diversification while the former favours intensification. These policies either impose one array *Clone* for each thread or a global *Clone* array over all the threads. For convenience, these strategies are called:

- *LCD* (Local Clone Detection) when the clone detection is restricted to solutions of the current thread;
- *GCD* (Global Clone Detection) when the clone detection is done over all the solutions investigated by the threads.

Moreover, whatever the policy used (*GCD* or *LCD*), two memory management strategies are defined:

- a Long Term Memory Strategy (*LTMS*) in which the memory is only initialized at the beginning of the GRASP $\times$ parallel ELS. Such strategy favors diversification by preventing identical solution investigation during two different ELS iterations.
- a Short Term Memory Strategy (*STMS*) in which the memory is initialized at the end of each ELS loop. This favors intensification since two ELS iterations can lead to the exploration of similar regions.

Depending on the policy (*LCD* / *GCD*) and on the strategy (*LTMS* / *STMS*), the shared memory management creates a cooperation by indirectly controlling the behaviour of the search threads during optimization. It also helps balancing between diversification and intensification.

9.7 Computational Evaluation

As stressed before, parallelizing the search space investigation in combinatorial optimization problems should be made in order to both widen the search and enforce the convergence, in addition to speed up the algorithm with parallel calculation. In this ethic, our GRASP $\times$ parallel ELS takes advantage of multi-core processors trying to make the threads collaborate. This section provides a computational evaluation attesting whether the search strategy we promote does not only allow solving larger problems but also finding improved solutions (with respect to the sequential counterpart), and leading to a better convergence.

9.7.1 Settings and Benchmarks

Our method has been implemented in Pascal using Delphi 6.0 package. The numerical experiments were run on a 2.3 GHz AMD Opteron quad-cores computer running Windows 2003 with 256 Go of memory. Several sets of instances have been used. They can be divided into two main categories: classical HVRP instances and new HVRP instances.

9.7.1.1 Classical HVRP Instances

A classical set of HVRP instances has been used to assess the performance of the proposed parallel approach. It has been proposed by Taillard [19]. It is made of 8 small instances with 50 to 100 nodes. They correspond to the VFMP-V transformed into HVRP instances by limiting the vehicles availability.

We compare our results with 5 methods from the literature on these classical HVRP instances. The first approach is a heuristic column generation [19] proposed by the author [19]. Tarantilis et al. [14] designed a threshold accepting algorithm (TA). Li et al. [21] published a record-to-record (RTR) travel metaheuristic. Prins [12] proposed a memetic algorithm (SMA-D2). Finally, Brandão [4] developed a tabu search algorithm (TSA).

To perform a fair comparison between methods that have been implemented on different processors, Dongarra [25] provides scaling factors depending on the processor speed. Thus we scale all the times with respect to the computer used in this chapter. For each of the 5 previously mentioned methods, Table 9.2 gives the processor used, the processor speed in Mflops and the scale factor applied in the next Tables.

Table 9.2 Scale factors in numerical experiments

Publications	Computers	Mflops	Scale factor
Taillard [19]	50MHz Sun Sparc 10	27	0.006
Prins [12]	1.8GHz Pentium 4M	1564	0.347
Tarantilis et al. [14]	400Mhz Pentium II	262	0.058
Li et al. [21]	1GHz Athlon	1168	0.259
Brandão [4]	1.4GHz Pentium 4M	1216	0.270
Our work	2.1 GHz Opteron	4500	1.000

9.7.1.2 New Realistic HVRP Instances (DLP_HVRP)

To better evaluate the proposed method, we also use large size instances, denoted DLP_HVRP and available at <http://www.isima.fr/lacomme/hvrp/hvrp.html> and at

<http://prodhonc.free.fr/>. To the best of our knowledge, these instances are the first ones based on real life district data.

Using the GIS software developed by Bajart and Charles [49], one instance is built for each of the 96 French districts, and contains from 60 to more than 250 nodes. Nodes correspond to cities larger than 100 or 500 citizens. Costs between nodes are computed as the shortest paths using the Google web service and thus they correspond to the real distance in kilometers between cities. The fleet composition contains up to 10 types of vehicles and has been randomly generated. As a consequence, some node demand can exceed the capacity of some types of vehicles and it may also happen that the total fleet capacity is close to the total demand to service. In addition, both fixed and variable vehicle costs are not dependant of the vehicles capacity. Thus, for some instances the smallest vehicles have smallest fixed cost while for some others the smallest vehicles are the most expensive in terms of variable and/or fixed costs. These characteristics might lead to very hard instances and be challenging for the metaheuristics.

9.7.1.3 Parameters

To have a fair comparison, two sets of parameters are defined: one for the classical instances and another for the DLP_HVRP instances. The parameters are detailed in Tables 9.3 and 9.4.

Table 9.3 Parameter Definition

Parameter	Definition
mtl	Maximal labels generated in split
mlpn	Maximal labels saved on each node in split
Nls	Maximal number of iterations during local search
Ns	Number of ELS iterations
Nr	Maximal number of ELS without improvement
nd	Number of diversifications
Nc_max	Maximal number of unproductive iterations during mutation
Ne	Number of parallel ELS iterations
Nt	Number of threads
Np	Number of GRASP iterations
Nf	Number of solutions to save by each thread in F_i
Strategy	Dispatch strategy
Share Memory Policy	Clone detection strategy and memory management strategy

The main difference from the two settings is the maximal number of labels generated in split, set to 50 000 for the DLP instances since those instances are larger than the classical ones. For a similar reason, the number of labels per node is increased from 3 to 100.

Table 9.4 Parameter settings

Parameter	Classical HVRP instances	DLP_HVRP instances
mtl	10 000	50 000
mlpn	3	100
Nls	500	100
Ns	3	3
Nr	3	3
nd	10	10
Nc_max	10	10
Ne	15	15
Nt	from 2 to 32	from 2 to 32
Np	50	50
Nf	1.15	1.15
Strategy	DF	DF
Share Memory Policy	LCD + STMS	LCD + STMS

9.7.2 Evaluation of the Communication Time

Before focusing on the results obtained on the sets of instances, it would be interesting to check the communication time of our machine. Ideally, when increasing the number of threads from 1 to k in our framework, the total running time should remain the same (except small variations due to the synchronization feature). Indeed, the number of iterations does not depend on the number of threads. However, as already explained, if k threads, assigned to k cores, imply the use of a greater number of processors than for p threads, then a slowdown occurs. It is due to communication through buses. In our experiments with quad-core processors, this is observed with a factor r between 1 and 2 processors, $2r$ between 2 and 4 processors and $4r$ between 4 and 8 processors, with $r = 1.55$. If an extra convergence rate is obtained one can state it is induced by an effective dispatch, synchronization and shared memory management.

Thus, in the next Tables, Columns TT provide the average total computational time in seconds on the instances. The time in seconds to find the best solution is given by T^* and this value can be corrected (*Corrected T^**) as explained above by the value that T^* would have taken if the threads were associated to cores belonging to a same processor (no extra time due to communication through buses).

9.7.3 Results on Classical HVRP Instances

Table 9.5 provides a comparative study of the last publications using the set from Taillard. *BKS* refers to the value of the best-known solution for each instance. *Avg. dev* corresponds to the average gap from a considered method to BKS in percent. *Avg. time* is the average CPU time of the methods in seconds, and *Scaled time* provides the corresponding time that would be achieved on our computer using the

scaling factors from Table 9.2. The line *#Best* expresses the number of BKS achieved by the method. Asterisks indicate when the BKS is an optimal cost.

The best published method so far has been proposed by Li et al. [21] with a deviation to the best-known solutions lower than 0.04%. This method provides 7 times the best solutions. The last two publications of Brandão [4] and Prins [12] have a higher average deviation to the best-known solutions (about 0.08%) but they get 6 BKS (vs. 7 for Li et al.) and they are twice faster (scaled).

Table 9.5 Performances of the last published methods on the HVRP instances from Taillard

		Taillard [19]	Tarantilis [14]	Li [21]	Prins [12]	Brandão [4]
n	BKS	Cost	Cost	Cost	Cost	Cost
13	50	1517.84*	1518.05	1519.96	1517.84	1517.84
14	50	607.53*	615.64	611.39	607.53	607.53
15	50	1015.29*	1016.86	1015.29	1015.29	1015.29
16	50	1144.94*	1154.05	1145.52	1144.94	1144.94
17	75	1061.96*	1071.79	1071.01	1065.85	1061.96
18	75	1823.58*	1870.16	1846.35	1823.58	1831.36
19	100	1117.51	1117.51	1123.83	1120.34	1120.34
20	100	1534.17*	1559.77	1556.35	1534.17	1534.17
Avg. dev		0.9310	0.6172	0.0317	0.0774	0.0850
Avg. time		2011.12	607.12	285.75	94.74	151.66
Scaled time		12.06	35.34	74.16	32.92	40.98
# Best		1	1	7	6	6

Tables 9.6 reports the performance of the GRASP x parallel ELS with 1 to 32 threads with the same statistical indicators as in Table 9.5, except the times that are given in Table 9.7 providing the average CPU times to achieve the best found solution and the corrected time due to the communication between processors.

While the parameters have not been especially tuned on these instances, the initial performance (single thread) is at about 0.53% of the best-known solutions, which is already better than both Taillard and Tarantilis et al. proposals. However, the single thread performance is slower and it does not compete with Li et al., Brandão or Prins methods.

The conclusion is totally different when the number of threads increases. On these small instances, it clearly appears that going from 1 to 2 threads significantly improves the results. The average deviation drops to 0.05%, becoming 10 times lower than with a single thread, and the average time to get the best solution is halved. Thus our proposed approach with 2 threads is only outperformed by Li et al. in terms of solution cost. A first sign of a better convergence seems to appear.

Until 8 threads, no better solution is found (6 instances over 8 are already solved to optimality). Then, with 16 and 32 threads, the methods provides results of the same quality as the best published method of Li et al. with an average deviation of 0.0317%. Increasing the number of threads allows to enforce the search on the solution space thanks to the balanced diversification, intensification and cooperation.

Table 9.6 Performances of the GRASP $\times$ parallel ELS on the classical HVRP instances from Taillard

		1 Thread	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
n	BKS	Cost	Cost	Cost	Cost	Cost	Cost
13	50	1517.84*	1517.84	1517.84	1517.84	1517.84	1517.84
14	50	607.53*	609.17	607.53	607.53	607.53	607.53
15	50	1015.29*	1015.29	1015.29	1015.29	1015.29	1015.29
16	50	1144.94*	1144.94	1144.94	1144.94	1144.94	1144.94
17	75	1061.96*	1065.2	1064.07	1064.07	1061.96	1061.96
18	75	1823.58*	1823.58	1823.58	1823.58	1823.58	1823.58
19	100	1117.51	1120.34	1120.34	1120.34	1120.34	1120.34
20	100	1534.17*	1534.17	1534.17	1534.17	1534.17	1534.17
Avg. dev		0.5313	0.0565	0.0565	0.0565	0.0317	0.0317
# Best		5	6	6	6	7	7

Table 9.7 Times of the GRASP $\times$ parallel ELS on the classical HVRP instances from Taillard

		1 Thread	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
n		Time	Time	Time	Time	Time	Time
13	50	60.7	80.7	5.8	14.3	17.9	38.7
14	50	46.6	353.1	153.9	230.7	79.0	111.7
15	50	17.7	62.0	18.4	10.7	10.9	22.9
16	50	109.0	2.1	1.1	2.5	3.8	7.7
17	75	182.6	109.7	49.2	37.2	123.8	482.0
18	75	287.8	68.3	72.3	32.9	25.5	38.9
19	100	702.0	16.0	142.8	22.4	31.4	10.3
20	100	297.7	190.3	434.2	116.8	27.5	249.5
Avg. T^*		213.0	110.3	109.7	58.5	40.0	120.2
Avg. corrected T^*		213.0	110.3	109.7	37.7	12.9	19.4

However, to really attest the performance, both the solution cost and the computational time have to be observed to confirm the convergence brought by the key-features of the proposed method. Tables 9.7 helps us to focus on that aspect.

As already said, from 1 to 2 threads, both the solution cost and T^* decrease. The same observation is made between 8 and 16 threads, reinforcing our thought about a better convergence rate provided by a higher number of threads. Moreover with 8 and 16 threads, T^* reduces from around 110s to respectively 58s and 40s (the latter also coming with better results) instead of experiencing a slow-down.

The small increase of corrected CPU time happens with 32 threads. It can be explained by two reasons. The first one is the synchronization time that can be longer because of a large number of threads. The second one is the main ground. The management of the shared memory forbids the search on solution already visited among the threads. However, the solution space is not so large on small instances, and many solutions are discarded due to clone detection leading to many unproductive trials.

To conclude, our method is competitive with the literature on Taillard's set, even if this must be moderated since all the recent published methods provide less than

1% deviation. This study exposes the significant impact of number of threads on the convergence and therefore on the solution quality and the computational time. Next Section is dedicated to larger instances to confirm the behavior brought by the balanced diversification, intensification and cooperation between parallel calculations thanks to the key-features provided in our framework.

9.7.4 Results on DLP_HVRP Instances

These instances can be divided into the following subsets:

- DLP_HVRP_1: 15 small instances with less than 100 nodes;
- DLP_HVRP_2: 40 medium instances with 100 to 150 nodes;
- DLP_HVRP_3: 33 large instances with 150 to 200 nodes;
- DLP_HVRP_4: 11 very large instances with more than 200 nodes.

The two next Sections summarize the performance of our GRASP x parallel ELS. Detailed results per instance are provided in Appendix, in which Cost, T^* and TT respectively mean the solution cost, the time to achieve the best found solution and the total time of the method. The last line indicates the average values of the columns.

9.7.4.1 DLP_HVRP_1 Instances (Small Scale Instances with Less Than 100 Nodes)

This set encompasses 15 small-scale instances with less than 100 nodes. Table 9.8 summarizes the results obtained according to the number of threads. The meaning of the indicators are the same as in Tables from Section 9.7.3, and % Best expresses the percent number of BKS computed by the method.

Table 9.8 GRASP x parallel ELS performance with 1 to 32 threads on DLP_HVRP_1

# Threads	Avg. Cost	Avg. dev	# Best	% Best	Avg. T^*	Corrected Avg. T^*
1 thread	4391.37	0.11	7	46.7	109.1	109.1
2 threads	4389.96	0.10	7	46.7	92.6	92.6
4 threads	4390.30	0.10	8	53.3	105.9	105.9
8 threads	4387.30	0.05	9	60.0	182.6	117.8
16 threads	4386.14	0.03	11	73.3	193.4	62.4
32 threads	4386.10	0.01	13	86.7	409.0	66.0

The gap to the best-known solutions clearly decreases between 1 to 32 threads while the average T^* is halved. Both reductions on cost and on computational time illustrate an improvement on the convergence rate.

The quality of the solution is better stressed by an analysis of the percent of best-known solutions found as shown in column % Best of Table 9.8. Even if the

reduction of the average cost is not large (from 4391.37 to 4386.10), the number of best-known solutions found is doubled between a single thread version of the method and the parallelization on 32 threads.

9.7.4.2 DLP_HVRP_2, DLP_HVRP_3, DLP_HVRP_4 Instances

Table 9.9 sums up the results obtained on the three subsets of medium and large-scale DLP_HVRP instances. Once again, the quality of the solution costs is clearly improved with the number of threads used while the computational time increases but not more than the expected extension due to the communication between processors.

Table 9.9 Average solutions depending on the number of threads

DLP_HVRP_2						
# Threads	Avg.Cost	Avg. dev	# Best	% Best	Avg. T^*	Corrected Avg. T^*
1	8712.516	0.52	4	10.5	427.1	427.1
2	8704.628	0.43	3	7.9	413.8	413.8
4	8696.673	0.35	6	15.8	429.8	429.8
8	8684.732	0.22	9	23.7	389.8	251.5
16	8675.003	0.13	13	34.2	498.4	160.8
32	8669.608	0.05	26	68.4	794.9	128.2
DLP_HVRP_3						
# Threads	Avg.Cost	Avg. dev	# Best	% Best	Avg. T^*	Corrected Avg. T^*
1	11995.19	1.34	0	0.0	663.5	663.5
2	11954.47	0.99	1	3.2	693.3	693.3
4	11913.26	0.66	3	9.7	704.3	704.3
8	11897.53	0.52	4	12.9	704.8	454.7
16	11877.95	0.32	9	29.0	773.1	249.4
32	11856.20	0.14	16	51.6	1042.9	168.2
DLP_HVRP_4						
# Threads	Avg.Cost	Avg. dev	# Best	% Best	Avg. T^*	Corrected Avg. T^*
1	14543.41	1.29	0	0.0	899.1	899.1
2	14509.72	1.02	0	0.0	1063.0	1063.0
4	14404.31	0.55	1	8.3	983.7	983.7
8	14389.63	0.36	1	8.3	1138.1	734.3
16	14398.82	0.37	5	41.7	1171.2	377.8
32	14365.49	0.23	6	50.0	1466.4	236.5

The full analysis of the DLP_HVRP_1 subset (see Section 9.7.4.1), which encompasses small-scale instances with less than 100 nodes, defines a general trend which is confirmed over the 3 last subsets. Depending on the instances set of interest the percent of best-known solutions obtained varies from (Table 9.9):

- 10% with 1 thread to 68% with 32 threads for the instance set DLP_HVRP_2;
- 0% with 1 thread to 51% with 32 threads for the instance set DLP_HVRP_3;
- 0% with 1 thread to 50% with 32 threads for the instance set DLP_HVRP_4.

For the small-scale instances, the difference from the single thread program and the 32 threads program is significant with a variation from 46% to 86% which correspond to a factor 2 (see Section 3.3.1). This ratio is about 6 for the DLP_HVRP_2 subset of instances (from 10% to 68%).

The last subsets (3 and 4) are interesting since the single thread version got no best-known solution. For these large-scale instances, the improvement in percent of the number of best-known solutions found is impressive when increasing the number of threads. On DLP_HVRP_3 instances, it steadily grows from 2 to 32 threads. On DLP_HVRP_4 instances, it first grows for 4 threads. Then it increases to 41% before stabilizing. This behavior points out the relationship between the size of the solution space and the benefit of extra threads to search a good solution.

On the other hand, the corrected average T^* significantly decreases with the number of threads. The double impact brought by the cooperative parallelization (improvement of the solution costs plus reduction of the computational times) at-tests the efficiency of proposed algorithm. The last Section of this experimental part details the convergence rate provided by the multi-threads.

9.7.5 Convergence Rate of the Parallel ELS

From the results presented in the previous Sections, it seems clear that the parallelization proposed in this chapter, with its key-features lending intensification, diversification and cooperation, leads to a convergence rate that is improved not only because of the number of threads but also thanks to the threads policy management during ELS synchronization, dispatch and selection (see Section 9.4).

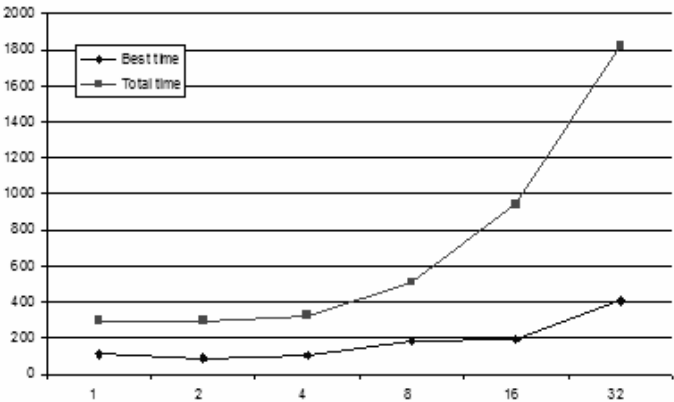


Fig. 9.3 Computational time and time to the best evolution depending on the number of threads - DLP_HVRP_1

The convergence rate is materialized by better results obtained within equivalent or shorter computational times (columns called Corrected Avg. T^* , see Tables 9.7, 9.8 and 9.9). Even when analysing the original value of T^* , one can note a quasi-linear evolution of the time to the best (T^*) and an exponential trend of the total time (see Figure 9.3).

The behavior of the convergence is also illustrated on Figure 9.4 representing the percentage of BKS found by the algorithm. The first set of instances (DLP_HVRP_1) seems to be easier to solve: the single thread approach is able to get 50% of the best-known solutions which can be compared with the 10% or 0% obtained for the last three sets. The general curve for the set 2, 3 and 4 seems to be piecewise linear especially for the last set.

We can now focus on the convergence through the evolution of the best found solution over the iterations. Comparative study is not straightforward since the

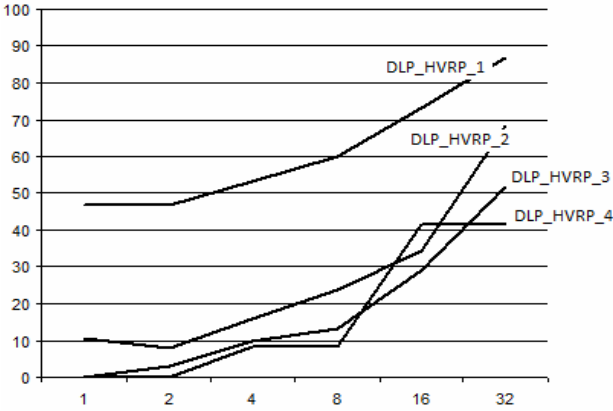


Fig. 9.4 Percents of best solutions depending on the number of threads - DLP_HVRP set

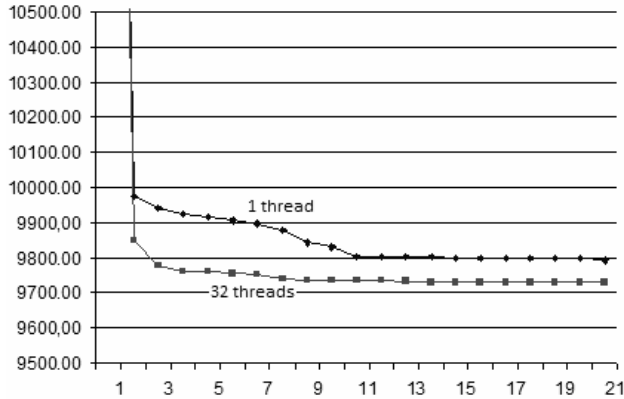


Fig. 9.5 Best solutions evolution with 1 and 32 threads

meta-heuristic is stochastic. Trying to analyze the relative impact of the number of threads, we introduce hereafter 5 runs with 1, 2, 4, 8, 16, 32 threads (for exactly the same parameters) for the largest example of the benchmark *i.e.* instance 18 with 256 nodes and 5 types of vehicles. Each run starts with the same initial solution at cost 13364.60.

After one iteration of the GRASP $\times$ parallel ELS, the best found solution varies from 10048 to 9849 with a strong advantage to the 32 threads run. Figure 9.5 gives the convergence curve over 20 iterations for 1 thread only and 32 threads. It tends to show that both the convergence rate and the solution quality are better when increasing the number of threads.

9.8 Concluding Remarks and Future Research

This article addresses a GRASP $\times$ parallel ELS framework for routing problems taking advantages of the multi-core processors technologies. They offer less parallel threads than computer cluster architectures for example (2 cores for entry point processors and 8 for upmarket processors) but they allow to benefit from the high speed memory and from the high processor frequencies. Large data accesses are convenient but the parallelisation is strongly limited and cannot be compared to the number of parallel threads offered in NVIDIA technology.

We introduce an algorithmic description of the GRASP $\times$ parallel ELS framework and we highlight the key-features including the thread management through intensification, diversification and cooperation. The GRASP $\times$ parallel ELS framework is based on both the efficient local search of [11] and the Split with Depth First Search Strategy [9].

Even if the key features are carefully set, increasing the number of threads may lead to an increased computational time. Indeed, assuming the same number of iterations per thread, the search should be enlarged proportionally to the number of threads while the computational time (from the user point of view) should remain the same. In practice, the delay of information exchange depends on the thread location. Exchange between threads located on two different processors requires the use of bus and is a lot more time consuming than data exchange within a single processor. Thus the communication is slower than information shared within a single processor. The phenomenon gets worse as the number of processors increases and overloading may appear if all the cores available on the computer are used.

Experiments on the heterogeneous vehicle routing problem show first that the GRASP $\times$ parallel ELS framework is of great interest taking advantages of a wide spread technology without any costly extra device like CUDA. Moreover, it shows that multi-threads can lead to extra-convergence rate and that better solutions can be obtained in the same computational time.

Future research is now directed towards new routing problems where the GRASP $\times$ parallel ELS framework could be applied especially for large-scale instances. This line of research could include the LRP with heterogonous fleet of vehicles.

References

1. Bortfeldt, A., Gehring, H., Mack, D.: A parallel tabu search algorithm for solving the container loading problem. *Parallel Computing* 29(5), 641–662 (2003)
2. Le Bouthillier, A., Crainic, T.G.: A cooperative parallel meta-heuristic for the vehicle routing problem with time windows. *Computers & Operations Research* 32(7), 1685–1708 (2005)
3. Tantar, A.-A., Melab, N., Talbi, E.-G., Parent, B., Horvath, D.: A parallel hybrid genetic algorithm for protein structure prediction on the computational grid. *Future Generation Computer Systems* 23(3), 398–409 (2007)
4. Brand, A.O.J.: A tabu search algorithm for the heterogeneous fixed fleet vehicle routing problem. *Computers & Operations Research* 38(1), 140–151 (2011)
5. Eksioglu, B., Vural, A.V., Reisman, A.: The vehicle routing problem: a taxonomic review. *Computers and Industrial Engineering* 57, 1472–1483 (2008)
6. Golden, B.L., DeArmon, J.S., Baker, E.K.: Computational experiments with algorithms for a class of routing problems. *Computers & Operations Research* 10(1), 47–59 (1983)
7. Duhamel, C., Lacomme, P., Prins, C., Prodhon, C.: A memetic approach for the capacitated location routing problem. In: *EU-MEeting 2008*, Troyes, France (2008)
8. Duhamel, C., Lacomme, P., Prins, C., Prodhon, C.: A GRASP_xELS approach for the capacitated location-routing problem. *Computers & Operations Research* 37(11), 1912–1923 (2010)
9. Duhamel, C., Lacomme, P., Prodhon, C.: Efficient frameworks for greedy split and new depth first search procedures for routing problems. *Computers & Operations Research* 38(4), 723–739 (2010)
10. Prins, C.: A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research* 31, 1985–2002 (2004)
11. Prins, C.: A GRASP *x* evolutionary local search hybrid for the vehicle routing problem. In: Pereira, F.B., Tavares, J. (eds.) *Bio-Inspired Algorithms for the Vehicle Routing Problem*. SCI, vol. 161, pp. 35–53. Springer (2009)
12. Prins, C.: Two memetic algorithms for heterogeneous fleet vehicle routing problems. *Engineering Applications of Artificial Intelligence* 22, 916–928 (2009)
13. Ribeiro, C.C., Rosseti, I.: Efficient parallel cooperative implementations of GRASP heuristics. *Parallel Computing* 33(1), 21–35 (2007)
14. Tarantilis, C.D., Kiranoudis, C.T., Vassiliadis, V.S.: A threshold accepting metaheuristic for the heterogeneous fixed fleet vehicle routing problem. *European Journal of Operational Research* 152, 148–158 (2004)
15. Fiechter, C.N.: A parallel tabu search algorithm for large traveling salesman problems. *Discrete Applied Mathematics* 51(3), 243–267 (1994)
16. Lourenço, H., Martin, O., Stitzle, T.: Iterated local search. In: Glover, F., Kochenberger, G. (eds.) *Handbook of Metaheuristics*, pp. 321–353. Kluwer, Dordrecht (2003)
17. Bertsekas, D.P., Tsitsiklis, J.N., Wu, C.: Rollout algorithms for combinatorial optimization problems. *Journal of Heuristics* 3, 245–262 (1997)
18. Talbi, E.-G., Cahon, S., Melab, N.: Designing cellular networks using a parallel hybrid metaheuristic on the computational grid. *Computer Communications* 30(4), 698–713 (2007)
19. Taillard, E.D.: A heuristic column generation method for the heterogeneous fleet VRP. *RAIRO Operations Research* 31(1), 1–14 (1999)
20. Guerriero, F., Mancini, M.: A cooperative parallel rollout algorithm for the sequential ordering problem. *Parallel Computing* 29(5), 663–677 (2003)
21. Li, F., Golden, B.L., Wasil, E.A.: A record-to-record travel algorithm for solving the heterogeneous fixed fleet vehicle routing problems. *Computers & Operations Research* 34, 2734–2742 (2007)
22. López, F.G., Torres, M.G., Batista, B.M., Pérez, J.A.M., Moreno-Vega, J.M.: Solving feature subset selection problem by a parallel scatter search. *European Journal of Operational Research* 169(2), 477–489 (2006)

23. Clarke, G., Wright, J.W.: Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research* 12, 568–581 (1964)
24. Berger, J., Barkaoui, M.: A parallel hybrid genetic algorithm for the vehicle routing problem with time windows. *Computers & Operations Research* 31, 2037–2053 (2004)
25. Dongarra, J.: Performance of various computers using standard linear equations software. Report CS-89-85. University of Tennessee (2009)
26. Cordeau, J.-F., Gendreau, M., Hertz, A., Laporte, G., Sormany, J.S.: New heuristics for the vehicle routing problem. In: Langevin, A., Riopel, D. (eds.) *Logistics Systems - Design and Optimization*, pp. 279–298. Springer (2005)
27. Beasley, J.E.: Route-first cluster-second methods for vehicle routing. *Omega* 11, 403–408 (1983)
28. Cadenas, J.M., Garrido, M.C., Muñoz, E.: Using machine learning in a cooperative hybrid parallel strategy of metaheuristics. *Information Sciences* 179(19), 3255–3267 (2009)
29. Current, J.R., Marsh, M.: Multiobjective transportation network design and routing problems: Taxonomy and annotation. *European Journal of Operational Research* 65, 4–19 (1993)
30. Sörensen, K.: Distance measures based on the edit distance for permutation-type representations. In: Barry, A. (ed.) *Proceedings of the Workshop on Analysis and Design of Representations and Operators (ADoRo)*, GECCO Conference, pp. 15–21 (2003)
31. Sörensen, K., Sevaux, M.: *MA | PM*: memetic algorithms with population management. *Computers & Operations Research* 33(5), 1214–1225 (2006)
32. Drummond, L.M.A., Ochi, L.S., Vianna, D.S.: An asynchronous parallel metaheuristic for the period vehicle routing problem. *Future Generation Computer Systems* 17(4), 379–386 (2001)
33. Ochi, L.S., Vianna, D.S., Drummond, L.M.A., Victor, A.O.: A parallel evolutionary algorithm for the vehicle routing problem with heterogeneous fleet. *Future Generation Computer System* 14, 285–292 (1998)
34. Desrochers, M.: An algorithm for the shortest path problem with resource constraints. Technical report G-88-27. GERAD (1988)
35. Toulouse, M., Crainic, T.G., Thulasiraman, K.: Global optimization properties of parallel cooperative search algorithms: A simulation study. *Parallel Computing* 26(1), 91–112 (2000)
36. Caricato, P., Ghiani, G., Grieco, A., Guerriero, E.: Parallel tabu search for a pickup and delivery problem under track contention. *Parallel Computing* 29, 631–639 (2003)
37. Lacomme, P., Prins, C., Ramdane-Chérif, W.: A Genetic Algorithm for the Capacitated Arc Routing Problem and Its Extensions. In: Boers, E.J.W., Gottlieb, J., Lanzi, P.L., Smith, R.E., Cagnoni, S., Hart, E., Raidl, G.R., Tijink, H. (eds.) *EvoASP 2001, EvoWorkshops 2001, EvoFlight 2001, EvoSTIM 2001, EvoCOP 2001, and EvoLearn 2001*. LNCS, vol. 2037, pp. 473–483. Springer, Heidelberg (2001)
38. Lacomme, P., Prins, C., Ramdane-Chérif, W.: Competitive memetic algorithms for arc routing problems. *Annals of Operations Research* 131, 159–185 (2004)
39. Toth, P., Vigo, D.: An overview of vehicle routing problems. In: *The Vehicle Routing Problem*, pp. 1–26. SIAM Monographs on Discrete Mathematics and Applications, Philadelphia (2002)
40. Baldacci, R., Battarra, M., Vigo, D.: Routing a heterogeneous fleet of vehicles. In: Wasil, E., Raghavan, S., Golden, B.L. (eds.) *The Vehicle Routing Problem: Latest Advances and New Challenges*. *Operations Research/Computer Science Interfaces*, vol. 43, pp. 3–27. Springer, Berlin (2008)
41. Wagner, R.A., Fischer, M.J.: The string-to-string correction problem. *Journal of the Association for Computing Machinery* 21, 168–173 (1974)
42. Ronald, S.: Distance functions for order-based encodings. In: Fogel, D. (ed.) *Proceedings of the IEEE Conference on Evolutionary Computation*, pp. 641–646 (1997)
43. Ronald, S.: More distance functions for order-based encodings. In: *Proceedings of the IEEE Conference on Evolutionary Computation*, pp. 558–563 (1998)

44. Porto, S.C.S., Kitajima, J.P.F.W., Ribeiro, C.C.: Efficient parallel cooperative implementations of GRASP heuristics performance evaluation of a parallel tabu search task scheduling algorithm. *Parallel Computing* 26(1), 73–90 (2000)
45. James, T., Rego, C., Glover, F.: A cooperative parallel tabu search algorithm for the quadratic assignment problem european. *Journal of Operational Research* 195(3), 810–826 (2009)
46. Feo, T.A., Resende, M.G.C.: Greedy randomized adaptive search procedures. *Journal of Global Optimization* 6, 109–133 (1995)
47. Crainic, T.G., Toulouse, M., Gendreau, M.: Parallel asynchronous tabu search for multicommodity location-allocation with balancing requirements. *Annals of Operations Research* 63, 277–299 (1996)
48. Cormen, T.H., Leiserson, C.L., Rivest, M.L.: *Introduction to algorithms*. MIT Press (1990)
49. Bajart, V., Charles, C.: *Systèmes d'information géographique*. 3rd Year Project Report. ISIMA (2009), <http://www.isima.fr/~lacomme/students.html>
50. Campos, V., Laguna, M., Martí, R.: Context-independent scatter and tabu search for permutation problems. *INFORMS Journal on Computing* 17, 111–122 (2005)
51. Bozejko, W.: Solving the flow shop problem by parallel programming. *Journal of Parallel and Distributed Computing* 69(5), 470–481 (2009)

Appendix

Table 9.10 DLP_HVRP.1 (small scale instances with less than 100 nodes)

	1 thread			2 threads			4 threads			8 threads			16 threads			32 threads		
	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*
HVRP_DLP.01	9219.65	466.5	290.3	9219.65	501.5	122.3	9219.65	449.6	24.7	9210.72	576.5	538.3	9210.14	760.2	587.1	9219.65	1494.7	14.6
HVRP_DLP.08	4597.35	281.6	277.4	4596.52	296.4	79.0	4596.52	278.5	40.4	4596.52	404.2	93.4	4596.52	766.7	29.7	4591.75	1528.1	450.1
HVRP_DLP.10	2107.55	220.3	10.9	2107.55	215.1	8.2	2107.55	247.6	3.7	2107.55	420.9	7.8	2107.55	952.3	3.6	2107.55	1783.1	45.6
HVRP_DLP.11	3370.47	298.9	43.9	3370.52	320.9	177.4	3369.91	310.5	264.6	3369.91	455.5	176.4	3369.91	859.8	64.2	3367.41	1632.1	624.8
HVRP_DLP.36	5721.84	378.1	377.2	5723.62	377.8	226.4	5719.79	346.8	286.6	5695.68	452.9	433.8	5684.61	715.9	276.3	5688.37	1452.8	1379.9
HVRP_DLP.39	2934.55	319.0	14.2	2934.55	325.3	104.4	2934.55	299.4	15.6	2934.55	417.2	19.8	2929.94	754.5	680.8	2923.72	1538.2	1449.5
HVRP_DLP.43	8753.95	579.5	134.3	8737.34	531.6	425.1	8749.01	485.1	311.6	8737.02	541.4	499.8	8737.34	743.6	214.0	8737.02	1464.4	110.1
HVRP_DLP.52	4027.27	296.5	161.5	4029.42	311.5	1.8	4027.27	305.7	161.5	4027.27	471.7	105.8	4027.27	886.2	200.5	4027.27	1767.2	105.7
HVRP_DLP.55	10244.34	201.4	127.1	10244.34	200.8	100.9	10244.34	244.4	5.3	10244.34	435.2	34.3	10244.34	879.4	27.5	10244.34	1745.5	30.1
HVRP_DLP.70	6692.91	354.3	103.5	6685.24	356.9	67.4	6685.24	353.8	164.8	6685.24	439.2	357.6	6685.24	741.5	25.5	6685.24	1506.8	30.7
HVRP_DLP.75	452.85	185.1	0.1	452.85	214.2	0.1	452.85	406.9	0.1	452.85	850.0	0.1	452.85	1786.1	0.1	452.85	3558.3	0.1
HVRP_DLP.82	4768.21	424.7	38.7	4768.21	342.0	24.1	4768.21	425.8	255.2	4768.21	811.5	284.4	4766.74	1402.2	499.1	4766.74	1912.3	1783.7
HVRP_DLP.92	564.39	172.0	1.7	564.39	181.5	0.9	564.39	256.0	1.0	564.39	536.7	0.1	564.39	1061.1	0.2	564.39	2188.5	0.5
HVRP_DLP.93	1036.99	128.8	43.5	1036.99	137.8	47.8	1036.99	195.6	51.6	1036.99	401.7	162.3	1036.99	833.7	286.4	1036.99	1679.9	99.8
HVRP_DLP.94	1378.25	150.3	12.1	1378.25	156.7	2.7	1378.25	231.03	2.3	1378.25	478.6	5.3	1378.25	1017.6	5.4	1378.25	2041.4	9.9
Avg.	4391.37	297.1	109.1	4389.96	298.0	92.6	4390.30	322.4	105.9	4387.29	512.9	182.6	4386.13	944.1	193.4	4386.10	1819.5	409.0

Table 9.11 DLP_HVRP_2 (medium scale instances with a number of nodes from 100 to 150)

	1 thread			2 threads			4 threads			8 threads			16 threads			32 threads		
	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*
HVRP-DLP-03	114.18.57	657.2	505.6	11424.08	732.8	641.9	11366.57	715.7	46.6	11350.48	748.0	424.4	11189.12	887.6	711.9	11201.68	1503.9	1027.0
HVRP-DLP-05	0940.02	757.2	699.7	0913.93	712.1	691.6	0908.67	703.9	576.4	0908.67	820.6	462.1	0906.93	984.5	748.0	0906.93	1563.5	1225.8
HVRP-DLP-06	11723.21	563.2	540.8	11742.55	986.8	235.1	11731.39	1112.0	877.7	11692.85	1233.5	926.3	11698.22	1566.3	577.2	11698.22	1940.6	1436.3
HVRP-DLP-07	8139.18	431.9	170.9	8118.44	580.1	552.7	8103.98	474.2	360.9	8095.88	633.0	627.4	8108.82	821.0	507.3	8101.80	1525.4	249.8
HVRP-DLP-12	3543.99	451.0	58.2	3543.99	491.6	22.9	3543.99	483.4	66.6	3543.99	553.5	25.4	3543.99	837.6	21.6	3543.99	1677.4	188.6
HVRP-DLP-13	6710.35	495.4	253.1	6700.57	546.6	198.8	6700.57	479.2	98.0	6697.58	590.5	135.7	6697.58	780.6	94.3	6696.43	1550.6	747.6
HVRP-DLP-16	4156.97	602.7	14.0	4156.97	732.5	88.7	4156.97	584.7	15.2	4156.97	784.7	13.1	4156.97	1021.1	24.2	4156.97	1792.5	20.0
HVRP-DLP-17	5367.34	591.1	424.1	5365.98	600.4	580.0	5367.34	580.9	168.3	5362.83	644.8	287.1	5365.97	806.7	564.0	5366.61	1515.7	294.5
HVRP-DLP-2A	7835.61	591.3	493.2	7854.58	693.9	396.8	7845.38	592.1	586.6	7798.92	851.1	321.9	7843.21	804.4	796.6	7793.21	1490.0	1374.4
HVRP-DLP-2B	8478.96	537.7	462.4	8479.19	655.0	651.3	8485.69	531.8	437.6	8471.12	587.0	383.0	8464.69	767.3	750.2	8467.73	1467.3	1022.2
HVRP-DLP-21	5150.05	570.6	476.6	5144.09	545.8	407.0	5147.27	549.7	491.1	5149.27	626.7	409.6	5144.76	787.4	525.9	5141.49	1521.8	538.1
HVRP-DLP-25	7215.01	695.7	534.7	7208.39	796.3	428.8	7212.68	711.2	427.8	7209.29	819.0	587.3	7209.29	974.3	414.9	7206.64	1614.1	778.0
HVRP-DLP-26	6486.99	482.4	475.7	6460.00	541.9	520.8	6453.99	2472.9	452.1	6509.14	569.7	259.4	6459.53	742.1	417.7	6446.31	1466.5	850.1
HVRP-DLP-28	5535.18	559.9	486.5	5540.73	642.7	607.0	5551.37	565.8	376.7	5537.87	689.9	633.6	5537.71	809.3	293.7	5531.06	1516.1	959.2
HVRP-DLP-30	6345.48	401.7	284.0	6334.18	458.0	435.9	6321.50	425.2	392.6	6329.88	475.2	408.9	6315.55	758.6	751.6	6313.39	1534.6	1020.0
HVRP-DLP-31	4123.83	584.2	167.2	4113.03	585.0	481.7	4116.72	645.5	413.0	4105.53	636.8	236.9	4110.18	819.1	101.2	4091.52	1575.2	1311.7
HVRP-DLP-34	5814.64	643.2	590.5	5823.80	631.0	487.7	5826.49	662.0	658.2	5786.73	711.9	153.6	5758.89	846.8	524.5	5768.76	1497.0	1201.5
HVRP-DLP-40	11211.61	711.3	476.8	11249.86	898.4	821.3	11186.59	924.1	716.7	11174.83	906.2	442.1	11148.07	1066.7	1032.6	11133.50	2433.8	922.5
HVRP-DLP-41	7633.72	724.9	633.4	7667.16	693.0	311.2	7647.67	660.6	658.8	7630.47	795.4	421.5	7651.70	884.0	305.3	7616.17	1788.1	1186.0
HVRP-DLP-47	16224.87	557.0	506.7	16218.32	703.9	602.2	16234.39	562.0	560.7	16210.56	615.8	454.1	16214.37	817.2	417.2	16206.14	1518.6	1195.0
HVRP-DLP-48	21370.88	531.6	346.8	21396.99	580.9	281.8	21359.57	548.8	130.7	21327.82	581.4	95.7	21318.04	775.1	722.1	21329.71	1496.4	207.4
HVRP-DLP-51	7798.40	717.0	678.8	7786.78	632.8	408.5	7794.67	629.2	578.4	7785.21	700.5	406.5	7774.52	824.6	487.5	7721.47	1506.2	545.1
HVRP-DLP-53	6435.24	636.3	522.0	6447.03	696.6	562.4	6455.53	629.8	478.4	6434.83	725.6	228.1	6435.24	838.4	461.8	6434.83	1549.4	773.4
HVRP-DLP-60	17037.39	665.3	614.9	17059.83	668.7	382.2	17079.64	659.3	429.7	17045.33	724.2	419.0	17045.33	882.0	322.2	17085.82	1543.0	608.0
HVRP-DLP-61	18681.83	500.7	294.3	18636.30	504.6	173.4	18636.30	471.1	160.8	18709.70	585.7	370.2	18580.30	768.2	63.2	18580.30	1487.5	1019.1
HVRP-DLP-66	13586.86	435.4	233.9	13412.58	523.8	85.2	13442.50	529.4	354.4	13383.27	620.3	591.7	13414.20	802.1	659.2	13301.34	1487.6	1449.8
HVRP-DLP-68	9174.21	656.6	595.7	9098.51	668.1	570.3	9114.35	628.4	501.2	8976.53	691.1	529.2	8988.82	848.2	812.4	9001.50	1478.0	1391.6
HVRP-DLP-73	10219.30	765.9	544.5	10241.86	843.1	441.5	10229.44	775.2	503.8	10212.69	799.6	443.8	10208.72	946.8	317.6	10206.05	1494.4	1490.8
HVRP-DLP-74	11695.40	626.5	299.6	11685.95	650.5	129.0	11634.84	659.5	592.1	11633.64	686.7	650.1	11614.37	856.7	771.4	11608.54	1513.4	1426.0
HVRP-DLP-79	7345.99	725.0	668.2	7273.67	762.3	756.1	7277.81	781.2	676.3	7262.91	756.0	244.0	7262.91	916.7	67.3	7259.54	1529.2	599.4
HVRP-DLP-81	1824.76	560.4	226.2	1824.76	651.7	19.5	1824.76	697.8	439.8	1824.68	739.2	50.2	1824.68	1026.5	281.0	1819.11	1715.5	503.6
HVRP-DLP-83	10027.91	598.5	550.4	10042.64	613.8	589.9	10019.15	617.6	561.3	10021.57	683.6	537.5	10019.15	867.0	732.9	10019.15	1553.1	1107.7
HVRP-DLP-84	7266.08	495.6	456.7	7266.08	592.3	62.0	7227.88	510.5	161.6	7229.48	1158.3	290.2	7227.88	1261.2	502.9	7227.88	2415.7	281.0
HVRP-DLP-85	8882.36	839.9	386.6	8864.67	950.7	551.9	8854.33	1243.5	447.1	8819.22	1564.9	859.9	8779.76	1830.3	1038.9	8815.20	6755.3	930.2
HVRP-DLP-87	3753.87	437.0	236.1	3753.87	469.4	20.5	3753.87	446.8	97.1	3753.87	487.8	38.1	3753.87	834.4	25.6	3753.87	1631.2	67.3
HVRP-DLP-88	12423.76	1025.7	951.0	12452.11	997.9	890.9	12402.85	949.8	482.4	12420.81	1069.0	482.2	12415.30	1255.0	986.5	12436.22	1723.3	110.8
HVRP-DLP-89	7136.36	680.5	344.1	7114.72	649.5	442.5	7112.05	642.8	364.4	7106.84	770.9	652.7	7109.06	894.5	729.1	7109.65	1488.4	800.2
HVRP-DLP-90	2359.42	344.6	25.1	2357.68	404.0	193.2	2347.81	368.9	101.8	2348.55	484.8	234.1	2346.43	870.5	378.3	2346.43	1674.1	344.7
Avg.	8712.51	601.4	427.1	8704.62	660.2	413.8	8696.67	637.5	429.8	8684.73	735.7	389.8	8675.00	906.6	498.4	8669.60	1750.9	794.9

Table 9.12 DLP_HVRP_3 (large scale instances with a number of nodes from 150 to 200)

	1 thread			2 threads			4 threads			8 threads			16 threads			32 threads		
	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*
HVRP_DLP.02	1230.67	1029.9	177.1	12116.07	1002.7	442.8	12243.68	1109.8	731.3	12096.78	1232.0	825.5	11982.15	1362.8	1185.4	12089.60	1617.6	1145.5
HVRP_DLP.04	11313.77	1010.5	101.0	11295.90	1121.4	986.2	11084.51	1267.7	910.0	11267.20	1187.7	425.2	11104.18	1371.2	907.7	11074.77	1608.3	861.7
HVRP_DLP.09	7653.11	715.4	553.6	7705.06	740.41	335.3	7681.72	885.6	829.0	7656.16	847.6	669.5	7633.90	1090.1	263.9	7619.19	1675.2	1097.2
HVRP_DLP.14	5672.28	917.0	824.1	5693.09	771.8	744.2	5644.98	903.0	601.6	5648.92	941.5	661.9	5670.52	982.8	886.1	5658.26	1556.0	1333.6
HVRP_DLP.15	8296.74	1089.0	1044.8	8290.65	1010.8	934.6	8236.40	1135.5	1078.5	8249.72	1178.6	924.5	8246.44	1298.0	1147.0	8251.40	1658.3	1320.3
HVRP_DLP.24	9194.39	821.1	810.6	9155.78	828.8	776.7	9156.73	899.1	857.6	9146.54	910.7	901.9	9101.47	1086.8	1014.6	9103.69	1553.8	1539.3
HVRP_DLP.29	9180.78	679.2	663.2	9144.50	677.1	711.3	9143.69	848.0	531.5	9147.23	951.6	497.0	9143.69	1056.3	402.7	9143.69	1506.3	402.7
HVRP_DLP.33	9483.53	1075.3	260.7	9443.30	1014.7	591.3	9488.53	1137.1	901.6	9517.93	1157.9	1080.7	9421.01	1178.3	697.4	9422.58	1620.7	1537.9
HVRP_DLP.35	9681.97	1102.1	1008.3	9623.06	1120.6	807.7	9622.31	1229.1	876.6	9641.28	1177.5	380.2	9601.03	1215.2	2005.4	9574.71	2061.7	1846.0
HVRP_DLP.37	6905.30	728.6	554.5	6904.01	729.1	632.9	6889.31	772.4	770.5	6882.27	807.7	644.3	6868.27	921.6	416.9	6858.23	1496.0	1221.8
HVRP_DLP.42	12092.69	782.5	741.5	11912.60	865.6	93.7	11809.31	883.8	655.6	11904.36	1007.5	619.8	11624.80	1140.7	1097.9	11658.94	1582.3	1511.2
HVRP_DLP.44	12480.47	891.8	597.4	12373.56	969.1	954.7	12350.71	1059.0	1012.4	12315.75	1061.9	583.2	12284.26	1177.9	394.5	12197.46	1654.4	1112.6
HVRP_DLP.45	10615.94	1106.8	1040.4	10587.68	1052.7	1036.3	10521.15	1238.2	1044.0	10537.58	1204.2	1055.4	10484.23	1371.7	661.0	10530.81	1586.3	838.1
HVRP_DLP.50	12453.04	722.6	518.1	12635.68	742.5	655.0	12481.23	830.0	816.5	12426.01	749.1	647.9	12459.49	928.0	846.2	12374.04	1528.2	589.0
HVRP_DLP.54	12505.92	544.7	239.4	11946.77	587.4	409.2	12030.81	633.9	172.7	11780.97	673.1	407.0	11809.07	908.1	463.7	11558.79	1517.5	916.1
HVRP_DLP.56	31228.55	770.0	741.5	31270.66	737.2	590.4	31152.02	768.4	616.6	31159.61	840.1	455.5	31129.76	968.6	519.4	31111.42	1500.1	602.4
HVRP_DLP.57	45103.90	901.8	869.9	45033.11	880.1	624.7	44837.58	1027.6	947.4	44818.52	1009.4	923.9	44818.18	1126.1	1123.5	44832.06	1691.7	1567.8
HVRP_DLP.59	14423.32	1100.3	750.8	14316.43	1038.5	895.3	14309.48	1230.5	366.5	14352.78	1211.5	1178.5	14374.43	1335.3	541.6	14378.29	1597.5	772.0
HVRP_DLP.63	20197.82	1020.7	1010.6	20136.76	1020.8	850.8	20150.83	981.6	660.4	20000.29	1072.7	1021.7	20075.70	1184.2	654.5	19951.76	1607.9	956.8
HVRP_DLP.64	19240.87	889.9	622.9	19232.11	791.3	377.9	19210.39	864.6	336.1	19202.47	865.3	469.1	19252.24	937.0	729.0	19202.47	1569.3	784.0
HVRP_DLP.67	11851.79	644.0	133.8	11906.52	713.5	509.9	11584.19	785.6	633.3	11555.73	828.2	268.1	11603.74	935.8	387.2	11483.25	1517.8	1297.1
HVRP_DLP.69	9309.46	908.4	712.5	9289.26	836.8	665.4	9222.10	897.4	676.6	9206.06	929.3	838.1	9162.78	1019.3	908.7	9198.22	1494.3	1189.2
HVRP_DLP.71	9936.25	904.6	817.8	9885.10	981.8	890.1	9891.70	1007.8	568.3	9892.86	1173.5	313.7	9870.22	1252.5	734.3	9873.41	1596.9	1557.6
HVRP_DLP.72	5989.28	1114.2	1096.7	5989.80	973.7	885.0	5975.63	1027.1	640.4	5923.25	1097.7	1004.6	5905.58	1183.9	1169.3	5920.67	1538.7	824.9
HVRP_DLP.76	12111.78	920.2	697.6	12132.31	920.5	789.2	12032.43	1037.1	564.2	12018.26	1044.6	727.5	12076.93	1179.5	914.9	12033.66	1538.1	371.6
HVRP_DLP.77	7005.73	975.2	676.6	6930.44	925.9	923.0	6960.52	957.4	943.8	6973.85	964.4	917.6	6952.56	1063.0	930.3	6947.52	1534.7	1127.1
HVRP_DLP.78	7054.8	962.4	792.6	7057.45	953.1	871.7	7058.02	980.8	705.1	7047.56	994.6	891.6	7040.79	1147.2	149.1	7035.01	1545.8	865.6
HVRP_DLP.80	6849.63	930.8	727.2	6864.41	887.7	885.1	6842.34	909.4	653.9	6839.92	939.1	593.7	6849.29	1127.1	811.6	6816.89	1568.4	478.7
HVRP_DLP.86	9065.30	714.6	578.8	9058.65	769.5	764.0	9060.11	740.6	326.5	9056.19	934.2	797.2	9051.72	1024.6	219.9	9030.68	1517.0	832.5
HVRP_DLP.91	6403.09	885.5	620.9	6413.77	934.1	900.8	6403.08	1034.0	775.3	6379.46	1040.4	817.4	6381.77	1224.3	677.9	6377.48	1649.4	1118.8
HVRP_DLP.95	1242.97	786.3	585.0	6244.13	734.5	381.5	6235.22	838.9	784.8	6181.60	819.1	273.2	6232.80	1018.0	1009.6	6233.21	1605.2	711.0
Avg.	11995.19	894.7	663.5	11954.47	881.7	693.3	11913.26	960.8	704.3	11897.53	991.9	704.8	11877.95	1148.8	773.1	11856.20	1590.2	1042.9

Table 9.13 DLP_HVRP_4 (strongly large scale instances with a number of nodes greater than 200)

	1 thread			2 threads			4 threads			8 threads			16 threads			32 threads		
	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*	Cost	TT	T*
HVRP-DLP_18	9792.66	1569.2	1518.4	9752.84	1337.2	1141.7	9768.43	1483.3	1238.9	9736.63	1830.2	1710.7	9704.38	1653.0	1137.0	9702.75	1961.2	1903.1
HVRP-DLP_19	11895.84	1207.8	1141.3	11909.97	1087.0	907.0	11869.61	1207.5	1072.4	11750.93	1300.3	1156.4	11839.38	1560.3	1534.6	11702.77	1692.5	1582.5
HVRP-DLP_22	13227.74	1486.1	741.8	13182.24	1531.16	1427.8	13120.01	1645.5	642.9	13115.59	1626.4	1293.3	13068.03	1784.1	996.3	13172.25	2548.0	2526.1
HVRP-DLP_23	7786.12	1326.5	780.5	7786.54	1353.7	1171.4	7804.05	1424.1	1060.6	7782.44	1513.4	875.4	7750.27	1755.8	449.6	7763.12	1813.5	1628.6
HVRP-DLP_27	8575.58	917.0	685.4	8535.36	989.2	947.7	8511.74	1063.5	1049.8	8484.22	1128.1	535.2	8485.90	1352.9	1261.3	8469.19	1685.6	1592.5
HVRP-DLP_32	9510.72	1444.6	1199.0	9464.42	1485.8	1363.6	9493.27	1557.2	1096.9	9417.62	1565.6	1090.8	9478.74	1848.8	1340.9	9430.57	1981.7	1697.4
HVRP-DLP_38	11338.71	941.0	852.7	11275.69	1011.5	928.1	11253.47	1030.4	906.1	11327.79	1059.2	1026.6	11242.95	1200.0	1194.2	11299.01	1556.9	1424.3
HVRP-DLP_46	25299.34	877.2	836.5	25213.41	984.7	978.2	24734.47	1326.4	1094.9	24741.84	1389.3	1331.8	24809.99	1503.0	1333.2	24674.26	1808.5	1585.3
HVRP-DLP_49	16726.72	820.9	701.7	16596.06	1046.6	1023.8	16530.99	969.4	726.0	16435.91	1047.4	756.8	16377.69	1285.2	1172.9	16394.64	1621.9	1225.8
HVRP-DLP_58	23654.21	1524.6	509.1	23777.31	1448.6	1223.9	23397.76	1634.8	1338.9	23400.96	1643.4	1634.0	23538.34	1764.1	1718.9	23492.93	1805.2	894.4
HVRP-DLP_62	23595.34	1162.9	1125.4	23549.89	1372.8	1288.7	23268.30	1263.3	1018.0	23415.10	1525.4	1060.6	23433.62	1820.0	1161.8	23149.61	1692.4	1075.2
HVRP-DLP_65	13117.99	1076.8	697.4	13072.92	1121.2	354.0	13099.59	1136.3	559.1	13066.55	1210.3	1185.7	13056.53	1435.8	753.9	13134.81	1634.4	461.5
Avg.	14543.41	1196.2	899.1	14509.72	1230.8	1063.0	14404.31	1311.8	983.7	14389.63	1403.3	1138.1	14398.82	1580.2	1171.2	14365.49	1816.8	1466.4